From: AAAI-02 Proceedings. Copyright © 2002, AAAI (www.aaai.org). All rights reserved.

A New Algorithm for Optimal Bin Packing

Richard E. Korf
Computer Science Department
University of California, Los Angeles
Los Angeles, CA 90095
korf@cs.ucla.edu

Abstract

We consider the NP-complete problem of bin packing. Given a set of numbers, and a set of bins of fixed capacity, find the minimum number of bins needed to contain all the numbers, such that the sum of the numbers assigned to each bin does not exceed the bin capacity. We present a new algorithm for optimal bin packing. Rather than considering the different bins that each number can be placed into, we consider the different ways in which each bin can be packed. Our algorithm appears to be asymptotically faster than the best existing optimal algorithm, and runs more that a thousand times faster on problems with 60 numbers.

Introduction and Overview

Given a set of numbers, and a fixed bin capacity, the bin-packing problem is to assign each number to a bin so that the sum of all numbers assigned to each bin does not exceed the bin capacity. An optimal solution to a bin-packing problem uses the fewest number of bins possible. For example, given the set of elements 6, 12, 15, 40, 43, 82, and a bin capacity of 100, we can assign 6, 12, and 82 to one bin, and 15, 40, and 43 to another, for a total of two bins. This is an optimal solution to this problem instance, since the sum of all the elements (198) is greater than 100, and hence at least two bins are required.

Optimal bin packing one of the classic NP-complete problems (Garey & Johnson 1979). The vast majority of the literature on this problem concerns polynomial-time approximation algorithms, such as first-fit and best-fit decreasing, and the quality of the solutions they compute, rather than optimal solutions. We discuss these approximation algorithms in the next section.

The best existing algorithm for optimal bin packing is due to Martello and Toth (Martello & Toth 1990a; 1990b). We present a new algorithm for optimal bin packing, which we call *bin completion*, that explores a different problem space, and appears to be asymptotically faster than the Martello and Toth algorithm. On problems of size 60, bin completion runs more than a thousand times faster than Martello and Toth's algorithm. We are able to optimally solve problems with 90 elements in an average of 2.5 seconds per problem.

Copyright © 2002, American Association for Artificial Intelligence (www.aaai.org). All rights reserved.

Approximation Algorithms

A simple approximation algorithm is called first-fit decreasing (FFD). The elements are sorted in decreasing order of size, and the bins are kept in a fixed order. Each element is placed into the first bin that it fits into, without exceeding the bin capacity. For example, given the elements 82, 43, 40, 15, 12, 6, and a bin capacity of 100, first-fit decreasing will place 82 in the first bin, 43 in a second bin, 40 in the second bin, 15 in the first bin, 12 in the second bin, and 6 in a third bin, for a total of three bins, which is one more than optimal.

A slightly better approximation algorithm is known as best-fit decreasing (BFD). It also sorts the elements in decreasing order of size, but puts each element into the *fullest* bin in which is fits. It can be implemented by keeping the bins sorted in increasing order of their remaining capacity, and placing each element into the first bin in which it fits. For example, given the set of elements 82, 43, 40, 15, 12, 6, best-fit decreasing will place 82 in bin a, 43 in bin b, 40 in bin b, 15 in bin b, because it is fuller than bin a, 12 in bin a, and 6 in bin a, for a total of two bins, which is optimal.

Both FFD and BFD can be implemented in $O(n \log n)$ time, but are not guaranteed to return optimal solutions. However, either algorithm is guaranteed to return a solution that uses no more than 11/9 of the optimal number of bins (Johnson 1973). On average, BFD performs slightly better than FFD. For example, on problems of 90 elements, where the elements are uniformly distributed from zero to one million, and the bin capacity is one million, BFD uses an average of 47.732 bins, while FFD uses an average of 47.733 bins. On these same problem instances, the optimal solution averages 47.680 bins. The FFD solution is optimal 94.694% of the time, and the BFD solution is optimal 94.832% of the time on these problem instances.

Optimal Solutions

Given the high quality solutions returned by these approximation algorithms, why bother trying to find optimal solutions? There are at least four reasons. In some applications, it may be important to have optimal solutions. In particular, with small numbers of bins, even a single extra bin is relatively expensive. In addition, being able to determine the optimal solutions to problem instances allows us to more accurately gauge the quality of approximate solutions. For example, the above comparisons of FFD and BFD solutions to

optimal solutions were only possible because we could compute the optimal solutions. Another reason is that an anytime algorithm for finding optimal solutions, such as those presented in this paper, can make use of any additional time available to find better solutions than those returned by BFD or FFD, which run very fast in practice. Finally, optimal bin packing is an interesting computational challenge, and may lead to insights applicable to other problems.

Estimated Wasted Space or L2 Lower Bound

A lower bound function for bin packing takes a problem instance, and efficiently computes a lower bound on the minimum number of bins needed. If we find a solution that uses the same number of bins as a lower bound, then we know that solution is optimal, and we can terminate the search.

An obvious lower bound is to sum all the elements, divide by the bin capacity, and round up to the next larger integer. A better bound starts with the sum of the elements, and adds an estimate of the total bin capacity that must be wasted in any solution, before dividing by the bin capacity. This is the L2 bound of Martello and Toth (Martello & Toth 1990a; 1990b), but we give a simpler and more intuitive algorithm for computing it below.

For example, consider the set of elements 99, 97, 94, 93, 8, 5, 4, 2, with a bin capacity of 100. There is no element small enough to go in the same bin as the 99, so that bin will have one unit of wasted space in any solution. The only element small enough to go in the same bin as the 97 is the 2, so the 2 can be placed with the 97 without sacrificing optimality, leaving a second unit of wasted space.

There are two remaining elements that could be placed with the 94, the 5 or the 4. In reality, only one of these elements could be placed with the 94, but to make our lower-bound calculation efficient and avoid any branching, we assume that we can place as much of their sum (9) as will fit in the bin with the 94, or 6 units. Thus, we assume there is no wasted space in this bin, and 3 units are carried over to the next bin, which contains the 93. The sum of all elements less than or equal to the residual capacity of this bin (7) is just the 3 units carried over from the previous bin. Therefore, at least $7 - 3 = 4$ additional units will be wasted between this bin and the previous one. Finally, there are no remaining elements to be placed with the 8, so 92 units must be wasted in this bin. Thus, the total wasted space will be at least $1 + 1 + 4 + 92 = 98$ units, which is added to the sum of all the elements before dividing by the bin capacity.

This estimated wasted-space calculation proceeds as follows. We consider the elements in decreasing order of size. Given an element x, the residual capacity r of the bin containing x is $r = c - x$, where c is the bin capacity. We then consider the sum s of all elements less than or equal to r, which have not already been assigned to a previous bin. There are three possible cases. The first is that r equals s. In that case, there is no estimated waste, and no carry over to the next bin. If $s < r$, then $r - s$ is added to the estimated waste, and again there is no carryover to the next bin. Finally, if $r < s$, then there is no waste added, and $s - r$ is carried over to the next bin. Once the estimated waste is computed, it is added to the sum of the elements, which is divided by the bin capacity, and then rounded up.

This estimated wasted space adds significantly to the sum of the elements. For example, on problems with 90 elements, uniformly distributed from zero to one million, with a bin capacity of one million, the estimated wasted-space lower bound averages 47.428 bins, while the simple sum lower bound averages only 45.497. For comparison, the average optimal solution for these problem instances is 47.680.

Martello and Toth Algorithm

The best existing algorithm for finding optimal solutions to bin-packing problems is due to Martello and Toth (Martello & Toth 1990a; 1990b). Their branch-and-bound algorithm is complex, and we describe here only the main features. Their basic problem space takes the elements in decreasing order of size, and places each element in turn into each partially-filled bin that it fits in, and into a new bin, branching on these different alternatives. This results in a problem space bounded by $n!$, where n is the number of elements, but this is a very pessimistic upper bound, since many elements won't fit in the same bins as other elements.

At each node of the search tree, Martello and Toth compute the first-fit, best-fit, and worst-fit decreasing completion of the corresponding partial solution. A partial solution to a bin-packing problem is one where some but not all elements have already been assigned to bins. A completion of a partial solution takes the current partially-filled bins, and assigns the remaining unassigned elements to bins. The worst-fit decreasing algorithm places each successive element in the partially-filled bin with the largest residual capacity that will accommodate that value.

Each of these approximate solutions is compared to a lower bound on the remaining solution that they call L3. The L3 bound is computed by successively relaxing the remaining subproblem by removing the smallest element, and then applying the L2 bound to each of the relaxed instances, returning the largest such lower bound. Martello and Toth's L2 bound equals the estimated wasted-space bound described above, but they use a more complex algorithm to compute it. If the number of bins used by any of the approximate completions equals the lower bound for completing the corresponding partial solution, no further search is performed below that node. If the number of bins in any approximate solution equals the lower bound on the original problem, the algorithm terminates, returning that solution as optimal.

The main source of efficiency of the Martello and Toth algorithm is a method to reduce the size of the remaining subproblems, which we will discuss below under "Dominance Relations". First, however, we very briefly discuss another optimal bin-packing program due to Fekete and Schepers.

Fekete and Schepers Lower Bound

Fekete and Schepers (Fekete & Schepers 1998) use the same algorithm as Martello and Toth, but with a more accurate lower-bound function. They claim that the resulting algorithm outperforms Martello and Toth's algorithm, based on

solving more problems with the same number of node generations, but didn't report running times. We implemented both algorithms, but found that the Fekete and Schepers algorithm was slower than that of Martello and Toth, because their lower bound function took longer to compute, and this wasn't compensated for by the reduced node generations.

Dominance Relations

Some sets of elements assigned to a bin are guaranteed to lead to solutions that are at least as good as those achievable by assigning other sets of elements to the same bin. We begin with some simple examples of these dominance relations, and then consider the general formulation.

First, consider two elements x and y, such that $x + y = c$, where c is the bin capacity. Assume that in an optimal solution, x and y are in different bins. In that case, we can swap y with all other elements in the bin containing x, without increasing the number of bins. This gives us a new optimal solution with x and y in the same bin. Thus, given a problem with two values x and y such that $x + y = c$, we can always put x and y in the same bin, resulting in a smaller problem (Gent 1998). Unfortunately, this does not extend to three or more elements that sum to exactly the bin capacity.

As another example, consider an element x such that the smallest two remaining elements added to x will exceed c. In other words, at most one additional element can be added to the bin containing x. Let y be the largest remaining element such that $x + y \leq c$. Then, we can place y in the same bin as x without sacrificing solution quality. The reason is that if we placed any other single element z with x, then we could swap y with z, since $z \leq y$ and $x + y \leq c$.

As a final example, assume that y is the largest remaining element that can be added to x such that $x+y \leq c$, and that y equals or exceeds the sum of *any* set of remaining elements that can be added to x without exceeding c. In that case, we can again put x and y in the same bin, without sacrificing solution quality. The reason is that any other set of elements that were placed in the same bin as x could be swapped with y without increasing the number of bins.

The general form of this dominance relation is due to Martello and Toth (Martello & Toth 1990a; 1990b). Define a *feasible* set as any set of elements whose sum doesn't exceed the bin capacity. Let A and B be two feasible sets. If all the elements of B can be packed into a set of bins whose capacities are the elements of A, then set A *dominates* set B. Given all the feasible sets that contain a common element x, only the undominated sets need be considered for assignment to the bin containing x.

Martello and Toth use this dominance relation to reduce the size of subproblems generated by their search. Given a partially-solved problem where some elements have already been assigned to bins, Martello and Toth first convert the partially-solved problem to an equivalent initial problem, where no elements have been assigned to bins, in two steps. First, any element that is unassigned, or assigned to a bin with no other elements, is left unchanged. Second, for each bin that contains more than one element, all the elements assigned to that bin are replaced with a single element equal to their sum, guaranteeing that any elements assigned to the same bin will stay together.

To reduce the size of such a problem, Martello and Toth take each element x in turn, starting with the largest element, and check to see if there is a *single* set of three or fewer elements, including x, that dominates all feasible sets containing x. If so, they place x with those elements in the same bin, and recursively apply the same reduction algorithm to the resulting reduced problem. They also use dominance relations to prune some placements of elements into bins.

Bin-Completion Algorithm

Our optimal bin-packing algorithm, which we call *bin completion*, makes more effective use of these dominance relations. Like the Martello and Toth algorithm, bin completion is also a branch-and-bound algorithm, but searches a different problem space. Rather than considering each element in turn, and deciding which bin to place it in, we consider each bin in turn, and consider the undominated feasible sets of elements that could be used to complete that bin. We sort the elements in decreasing order of size, and consider the bins containing each element in turn, enumerating all the undominated completions of that bin, and branching if there are more than one. In other words, we first complete the bin containing the largest element, then complete the bin containing the second largest element, etc.

Example Problem

To illustrate our algorithm, consider an example problem consisting of the elements {100, 98, 96, 93, 91, 87, 81, 59, 58, 55, 50, 43, 22, 21, 20, 15, 14, 10, 8, 6, 5, 4, 3, 1, 0}, with a bin capacity of 100. The 100 completely occupies a bin, and the 0 takes no space, resulting in the problem {98, 96, 93, 91, 87, 81, 59, 58, 55, 50, 43, 22, 21, 20, 15, 14, 10, 8, 6, 5, 4, 3, 1}. The only element that can go with 98 is 1, so we put them together, leaving {96, 93, 91, 87, 81, 59, 58, 55, 50, 43, 22, 21, 20, 15, 14, 10, 8, 6, 5, 4, 3}. We place 96 and 4 together, since they sum to exactly the bin capacity, leaving {93, 91, 87, 81, 59, 58, 55, 50, 43, 22, 21, 20, 15, 14, 10, 8, 6, 5, 3}. Since the sum of the two smallest remaining elements, 5 and 3, exceeds the residual capacity of the bin containing 93, we can't place more than one element in that bin, and choose the largest such element 6 to go with 93,, leaving {91, 87, 81, 59, 58, 55, 50, 43, 22, 21, 20, 15, 14, 10, 8, 5, 3}. We can complete the bin containing 91 with 8, 5, 3, or $5 + 3$. Since the 8 dominates the other single elements, and also the set $5 + 3$, we place 8 with 91, leaving {87, 81, 59, 58, 55, 50, 43, 22, 21, 20, 15, 14, 10, 5, 3}. The bin containing 87 can be completed with 10, 5, 3, $10 + 3$ or $5 + 3$. All of these are dominated by $10 + 3$, so we place $10 + 3$ with 87, resulting in {81, 59, 58, 55, 50, 43, 22, 21, 20, 15, 14, 5}. Up to this point, no branching is needed, since there is only one undominated choice for completing each of the bins considered so far.

The bin containing 81 can be completed with 15, 14, 5, or $14 + 5$. Both 15 and $14 + 5$ dominate both 14 and 5 individually. However, neither 15 nor $14 + 5$ dominate each other, so both must be considered, producing a two-way branch in